CodeMate — All-in-One Pipeline## Cell 0: Setup (run every restart)Mounts Drive, installs deps, sets paths.

```
from google.colab import drive
drive.mount('/content/drive')
import os
import json
import random
import re
import time
import subprocess
import tempfile
from pathlib import Path
from tqdm import tqdm

DRIVE_ROOT = "/content/drive/MyDrive/codemate"
DATA_DIR = f"{DRIVE_ROOT}/data"
CKPT_DIR = f"{DRIVE_ROOT}/checkpoints"
FINAL_DIR = f"{DRIVE_ROOT}/final_adapter"
RESULTS_DIR = f"{DRIVE_ROOT}/results"

for d in [DATA_DIR, CKPT_DIR, FINAL_DIR, RESULTS_DIR]:
    os.makedirs(d, exist_ok=True)

!pip install -q torch transformers>=4.40.0 peft>=0.10.0
bitsandbytes>=0.43.0 trl>=0.8.0 datasets accelerate>=0.30.0
sentencepiece protobuf scipy sacrebleu rouge-score gradio google-
generativeai codebleu
print("□ Setup done")
```

```
Mounted at /content/drive
□ Setup done
```

Phase 1: Data PreparationDownloads CodeAlpaca + python_code_instructions from HuggingFace.Saves `train.jsonl` to Drive. **Skip this cell if already done.**

```
# PHASE 1 — RUN THIS TO PREPARE DATA (skip if already done)
SYSP = "You are CodeMate, an AI code assistant. Analyze the following
code. If there are errors or tracebacks, identify the bug and suggest
a corrected version. If the code is functional, explain its behavior
step-by-step."
def fmt_debug(code, err, fix, expl):
```

```
    inp = f"<CODE>\n{code.strip()}\n</CODE>"
    if err: inp += f"\n<ERROR>\n{err.strip()}\n</ERROR>"
    return {"system":SYSP,"instruction":inp,"response":f"**Mode:
DEBUG**\n\n{expl}\n\n**Fixed:**\n```\n{fix.strip()}\
n```","task_type":"debug"}

def fmt_explain(code, expl):
    return {"system":SYSP,"instruction":f"<CODE>\n{code.strip()}\
n</CODE>","response":f"**Mode: EXPLAIN**\n\
n{expl}","task_type":"explain"}

from datasets import load_dataset

all_ex = []
# Source 1: CodeAlpaca
ds = load_dataset("sahil2801/CodeAlpaca-20k", split="train")
ekw = ["explain","describe","what does","how does","step by
step","purpose"]
dkw = ["fix","bug","error","wrong","debug","issue","fail"]

for r in tqdm(ds, desc="CodeAlpaca"):
    il = r["instruction"].lower(); code = r.get("input","") or ""; out
= r.get("output","") or ""
    if not out or len(out)<30: continue
    if any(k in il for k in ekw) and code:
all_ex.append(fmt_explain(code, out))
    elif any(k in il for k in dkw):
all_ex.append(fmt_debug(r["instruction"]+"\n"+code, "", out,
r["instruction"]))
    elif code and len(code)>50: all_ex.append(fmt_explain(code, out))

# Source 2: Python instructions
try:
    ds2 = load_dataset("iamtarun/python_code_instructions_18k_alpaca",
split="train")
    for r in tqdm(list(ds2)[:2000], desc="PythonInstr"):
        out = r.get("output",""); prompt = r.get("prompt","")
        if out and len(out)>50: all_ex.append(fmt_explain(out, f"This
code implements: {prompt}"))
except: print("⚠ Skipped python_code_instructions")

random.seed(42); random.shuffle(all_ex)
n = len(all_ex); te = int(n*0.8); ve = te + int(n*0.1)

for name, data in [("train",all_ex[:te]),("val",all_ex[te:ve]),
("test",all_ex[ve:])]:
    p = f"{DATA_DIR}/{name}.jsonl"
    with open(p,"w") as f:
        for x in data: f.write(json.dumps(x,ensure_ascii=False)+"\n")
    print(f"□ {name}: {len(data)} → {p}")
```

```
print(f"Total: {n}")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"75b1602421d64ff59a1d2ab802d4e7ba","version_major":2,"version_minor":0}

{"model_id":"3809c189951a4008b625df945bb74931","version_major":2,"version_minor":0}

{"model_id":"b55a36a7ed7347d89ce265975e8d6690","version_major":2,"version_minor":0}

```
CodeAlpaca: 100%|██████████| 20022/20022 [00:00<00:00, 37153.40it/s]
```

{"model_id":"c588b2e9ebe84f3a859433b644a75802","version_major":2,"version_minor":0}

{"model_id":"3cbbba063150468e9565fffafe239ffd","version_major":2,"version_minor":0}

{"model_id":"01be2d0a5f8744cd81e1cd862abe889d","version_major":2,"version_minor":0}

```
PythonInstr: 100%|██████████| 2000/2000 [00:00<00:00, 569375.42it/s]

□ train: 3670 → /content/drive/MyDrive/codemate/data/train.jsonl
□ val: 458 → /content/drive/MyDrive/codemate/data/val.jsonl
□ test: 460 → /content/drive/MyDrive/codemate/data/test.jsonl
Total: 4588
```

 Load Phase 1 (run this instead if data already on Drive)

```
# LOAD PHASE 1 — skip data prep, just verify files exist
for s in ["train","val","test"]:
    p = f"{DATA_DIR}/{s}.jsonl"
    if os.path.exists(p):
        c = sum(1 for _ in open(p))
        print(f"  {s}: {c} samples")
    else: print(f"  {p} NOT FOUND — run Phase 1 first")
```

```
  train: 3670 samples
  val: 458 samples
  test: 460 samples
```

Phase 2: Synthetic Error InjectionInjects 6 bug types into clean Python code to create debug training pairs.**Skip if already done.**

```
# PHASE 2 — SYNTHETIC ERRORS
CLEAN = ['def fibonacci(n):\n    if n <= 0:\n        return 0\n
elif n == 1:\n        return 1\n    return fibonacci(n-1) +
fibonacci(n-2)','def binary_search(arr, target):\n    left, right = 0,
len(arr)-1\n    while left <= right:\n        mid = (left+right)//2\n
if arr[mid]==target: return mid\n        elif arr[mid]<target:
left=mid+1\n        else: right=mid-1\n    return -1','import math\
ndef is_prime(n):\n    if n<2: return False\n    for i in
range(2,int(math.sqrt(n))+1):\n        if n%i==0: return False\n
return True','def two_sum(nums, target):\n    seen={}\n    for i,num
in enumerate(nums):\n        c=target-num\n        if c in seen:
return [seen[c],i]\n        seen[num]=i\n    return []','def
flatten(lst):\n    res=[]\n    for x in lst:\n        if
isinstance(x,list): res.extend(flatten(x))\n        else:
res.append(x)\n    return res','def count_words(text):\n
words=text.lower().split()\n    wc={}\n    for w in words:\n
w=w.strip(".,!?;:")\n        if w: wc[w]=wc.get(w,0)+1\n    return
wc','def matrix_mult(A,B):\n    rA,cA,cB=len(A),len(A[0]),len(B[0])\n
res=[[0]*cB for _ in range(rA)]\n    for i in range(rA):\n        for
j in range(cB):\n            for k in range(cA): res[i][j]+=A[i]
[k]*B[k][j]\n    return res',]

def inject_wrong_op(c):
    if '==' in c:
        return c.replace('==','=',1),"SyntaxError","Used = instead of
=="
    return None

def inject_missing_return(c):
    lines=c.split('\n')
    ri=[i for i,l in enumerate(lines) if l.strip().startswith('return
```

```
')]
    if ri:
        idx=ri[-1]
        removed=lines[idx]
        nl=lines[:idx]+lines[idx+1:]
        return '\n'.join(nl),"Returns None",f"Missing:
{removed.strip()}"
    return None

def inject_missing_colon(c):
    m=re.search(r'^(s*(?:def|if|elif|else|for|while|class)\b.+):(\s*)
$',c,re.M)
    if m:
        return
c.replace(m.group(0),m.group(1)+m.group(2),1),"SyntaxError: expected
':'","Missing colon"
    return None

def inject_wrong_indent(c):
    lines=c.split('\n')
    ind=[(i,l) for i,l in enumerate(lines) if l.startswith('    ') and
l.strip()]
    if ind:
        idx,l=random.choice(ind)
        nl=lines.copy()
        nl[idx]=l.lstrip()
        return '\n'.join(nl),"IndentationError","Wrong indentation"
    return None

def inject_offbyone(c):
    m=re.search(r'range\\((\\d+)\\)',c)
    if m:
        v=m.group(1)
        nv=str(int(v)-1)
        return c.replace(f'range({v})',f'range({nv})',1),f"Off-by-
one",f"range({nv}) should be range({v})"
    return None

def inject_missing_import(c):
    lines=c.split('\n')
    il=[(i,l) for i,l in enumerate(lines) if
l.strip().startswith(('import ','from '))]
    if il:
        idx,line=il[0]
        nl=[l for i,l in enumerate(lines) if i!=idx]
        return '\n'.join(nl),f"Missing import",f"Need: {line.strip()}"
    return None

strategies=[inject_wrong_op,inject_missing_return,inject_missing_colon
,inject_wrong_indent,inject_offbyone,inject_missing_import]
```

```
pairs = []
random.seed(42)
for _ in range(3000):
    code=random.choice(CLEAN)
    strat=random.choice(strategies)
    r=strat(code)
    if r and r[0].strip()!=code.strip():
        buggy,err,expl=r
        pairs.append({"system":SYSP,"instruction":f"<CODE>\n{buggy}\
n</CODE>\n<ERROR>\n{err}\n</ERROR>","response":f"**Mode: DEBUG**\n\
n{expl}\n\n**Fixed:**\n```python\n{code}\n```","task_type":"debug"})
pairs=pairs[:2000]
out=f"{DATA_DIR}/synthetic_debug.jsonl"
with open(out,"w") as f:
    for p in pairs: f.write(json.dumps(p,ensure_ascii=False)+"\n")
print(f"□ {len(pairs)} synthetic debug pairs → {out}")
```

```
□ 1774 synthetic debug pairs →
/content/drive/MyDrive/codemate/data/synthetic_debug.jsonl
```

□ Load Phase 2

```
p=f"{DATA_DIR}/synthetic_debug.jsonl"
if os.path.exists(p):
    print(f"□ Synthetic data: {sum(1 for _ in open(p))} pairs")
else:
    print("□ Not found — run Phase 2")
```

```
□ Synthetic data: 1774 pairs
```

Phase 3: QLoRA Fine-Tuning**~2 hours on T4.** Saves checkpoints to Drive every 200 steps.If session dies → restart runtime → run Cell 0 → skip to Cell 3b to resume.

Set Hugging Face Token

Ensure your Hugging Face token is set as a Colab secret named `HF_TOKEN`.

```
from google.colab import userdata
import os
from huggingface_hub import login

hf_token = None
try:
    # Try to get from Colab secrets first
    hf_token = userdata.get('HF_TOKEN')
except Exception as e:
    print(f"Could not load HF_TOKEN from Colab secrets: {e}")
```

```
if not hf_token:
    print("HF_TOKEN not found in Colab secrets. Please enter it
below:")
    hf_token = input("Hugging Face Token: ")

if hf_token:
    os.environ['HF_TOKEN'] = hf_token
    login(token=hf_token)
    print("□ Hugging Face token loaded and authenticated.")
else:
    print("□ Hugging Face token was not provided.")
```

```
Could not load HF_TOKEN from Colab secrets: Secret HF_TOKEN does not
exist.
HF_TOKEN not found in Colab secrets. Please enter it below:
Hugging Face Token: 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

Note: Environment variable`HF_TOKEN` is set and is the current active
token independently from the token you've just configured.
WARNING:huggingface_hub._login:Note: Environment variable`HF_TOKEN` is
set and is the current active token independently from the token
you've just configured.

□ Hugging Face token loaded and authenticated.
```

```
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"

import torch, gc
gc.collect()
torch.cuda.empty_cache()
torch.cuda.reset_peak_memory_stats()

# PHASE 3 — TRAINING
import torch
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig, TrainingArguments
from peft import LoraConfig, get_peft_model,
prepare_model_for_kbit_training
from trl import SFTTrainer

BASE_MODEL = "Qwen/Qwen2.5-Coder-1.5B-Instruct"
MAX_SEQ = 512

# Load all data
all_data = []
for fn in ["train.jsonl","synthetic_debug.jsonl"]:
    p = f"{DATA_DIR}/{fn}"
    if os.path.exists(p):
```

```
        with open(p) as f:
            all_data.extend([json.loads(l) for l in f if l.strip()])
        print(f"Loaded {fn}: {sum(1 for _ in open(p))}")

def fmt_chat(ex):
    return {"text": f"<start_of_turn>user\n{ex.get('system','')}\n\
n{ex['instruction']}<end_of_turn>\n<start_of_turn>model\
n{ex['response']}<end_of_turn>"}

ds = Dataset.from_list([fmt_chat(r) for r in all_data])
sp = ds.train_test_split(test_size=0.1, seed=42)
print(f"Train: {len(sp['train'])} | Val: {len(sp['test'])}")

# Model
bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True
)
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
tokenizer.pad_token=tokenizer.eos_token
tokenizer.padding_side="right"
model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=bnb,
    device_map="auto",
    torch_dtype=torch.bfloat16,
)
model = prepare_model_for_kbit_training(model)
lora = LoraConfig(
    r=16,
    lora_alpha=32,

target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_pr
oj","down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora)
tp=sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable: {tp:,} params | GPU:
{torch.cuda.memory_allocated()/1e9:.1f}GB /
{torch.cuda.get_device_properties(0).total_memory/1e9:.1f}GB")

from trl import SFTTrainer, SFTConfig
import inspect
import torch
from datasets import Dataset
```

```python
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig, TrainingArguments
args = TrainingArguments(
    output_dir=CKPT_DIR,
    num_train_epochs=3,
    per_device_train_batch_size=1,
    per_device_eval_batch_size = 1,
    gradient_accumulation_steps= 4,
    learning_rate=2e-4,
    weight_decay=0.01,
    warmup_ratio=0.01,
    lr_scheduler_type="cosine",
    bf16=True,
    logging_steps=25,
    save_steps=100,
    save_total_limit=3,
    eval_strategy="steps",
    eval_steps=100,
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    report_to="none",
    optim="paged_adamw_8bit",
    gradient_checkpointing=False,
    gradient_checkpointing_kwargs={"use_reentrant": False},
    dataloader_pin_memory=False,
    dataloader_num_workers=4,
    max_grad_norm=0.3,
    group_by_length=True,
)

sig = inspect.signature(SFTTrainer.__init__).parameters

tkw = {
    "model": model,
    "args": args,
    "train_dataset": sp["train"],
    "eval_dataset": sp["test"]
}

if "processing_class" in sig:
    tkw["processing_class"] = tokenizer
elif "tokenizer" in sig:
    tkw["tokenizer"] = tokenizer

for k, v in [
    ("max_seq_length", MAX_SEQ),
    ("dataset_text_field", "text"),
    ("packing", True)
]:
    if k in sig:
```

```
        tkw[k] = v

trainer = SFTTrainer(**tkw)

# Check for existing checkpoint
ckpts = sorted(Path(CKPT_DIR).glob("checkpoint-*"),
key=os.path.getmtime)
ckpt = str(ckpts[-1]) if ckpts else None

if ckpt:
    print(f"□ Resuming from {ckpt}")

result = trainer.train(resume_from_checkpoint=ckpt)
print(f"□ Training done! Loss: {result.training_loss:.4f} | Time:
{result.metrics.get('train_runtime',0)/60:.1f}min")
model.save_pretrained(FINAL_DIR)
tokenizer.save_pretrained(FINAL_DIR)
print(f"□ Adapter saved to {FINAL_DIR}")
```

```
Loaded train.jsonl: 3670
Loaded synthetic_debug.jsonl: 1774
Train: 4899 | Val: 545
```

{"model_id":"1fefc81fa8884d7b850dd82fb9f2ae5e","version_major":2,"version_minor":0}

{"model_id":"07eaa577b2e34735943ccd266f168086","version_major":2,"version_minor":0}

{"model_id":"18753136338e411bbefe091a581cd945","version_major":2,"version_minor":0}

{"model_id":"9b56d97e57b14ad2ad353564903f0070","version_major":2,"version_minor":0}

{"model_id":"0f779f17549c4f9cadc36f92702e2804","version_major":2,"version_minor":0}

```
`torch_dtype` is deprecated! Use `dtype` instead!
```

{"model_id":"520ce8c5be514d858ccca4cf8552f4dc","version_major":2,"version_minor":0}

{"model_id":"e0901008f3724408a6a029cc234c9847","version_major":2,"version_minor":0}

{"model_id":"4b979ae4a9f448eb9408e37d05a248f7","version_major":2,"version_minor":0}

```
warmup_ratio is deprecated and will be removed in v5.2. Use
`warmup_steps` instead.
```

```
warmup_ratio is deprecated and will be removed in v5.2. Use
`warmup_steps` instead.

Trainable: 2,179,072 params | GPU: 1.6GB / 15.6GB

{"model_id":"36cbf7fa37ca4c88af4a114afacc860f","version_major":2,"vers
ion_minor":0}

{"model_id":"ad88e0d79e58402aa106fb0afecac366","version_major":2,"vers
ion_minor":0}

{"model_id":"ce589dc7b55043589531852ef911cfde","version_major":2,"vers
ion_minor":0}

{"model_id":"f1602d9613fe4ceba40031887d435282","version_major":2,"vers
ion_minor":0}

The tokenizer has new PAD/BOS/EOS tokens that differ from the model
config and generation config. The model config and generation config
were aligned accordingly, being updated with the tokenizer's values.
Updated tokens: {'bos_token_id': None, 'pad_token_id': 151645}.

 Resuming from
/content/drive/MyDrive/codemate/checkpoints/checkpoint-1500

/usr/local/lib/python3.12/dist-packages/torch/utils/data/
dataloader.py:424: UserWarning: This DataLoader will create 4 worker
processes in total. Our suggested max number of worker in current
system is 2, which is smaller than what this DataLoader is going to
create. Please be aware that excessive worker creation might get
DataLoader running slow or even freeze, lower the worker number to
avoid potential slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
```

```
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)

<IPython.core.display.HTML object>

/usr/local/lib/python3.12/dist-packages/torch/_dynamo/
eval_frame.py:1181: UserWarning: torch.utils.checkpoint: the
use_reentrant parameter should be passed explicitly. Starting in
PyTorch 2.9, calling checkpoint without use_reentrant will raise an
exception. use_reentrant=False is recommended, but if you need to
preserve the current default behavior, you can pass
use_reentrant=True. Refer to docs for more details on the differences
between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
```

```
<IPython.core.display.HTML object>
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/
dataloader.py:424: UserWarning: This DataLoader will create 4 worker
processes in total. Our suggested max number of worker in current
system is 2, which is smaller than what this DataLoader is going to
create. Please be aware that excessive worker creation might get
DataLoader running slow or even freeze, lower the worker number to
avoid potential slowness/freeze if necessary.
```

```
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
```

```
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
```

```
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
```

```
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
```

```
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
```

```
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
```

```
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:424: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
```

```
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
81: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but if you need to preserve the
current default behavior, you can pass use_reentrant=True. Refer to
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)

 Training done! Loss: 0.1411 | Time: 295.3min
 Adapter saved to /content/drive/MyDrive/codemate/final_adapter
```

Phase 4: EvaluationComputes BLEU, ROUGE-L, CodeBLEU on test set.Saves results to Drive.

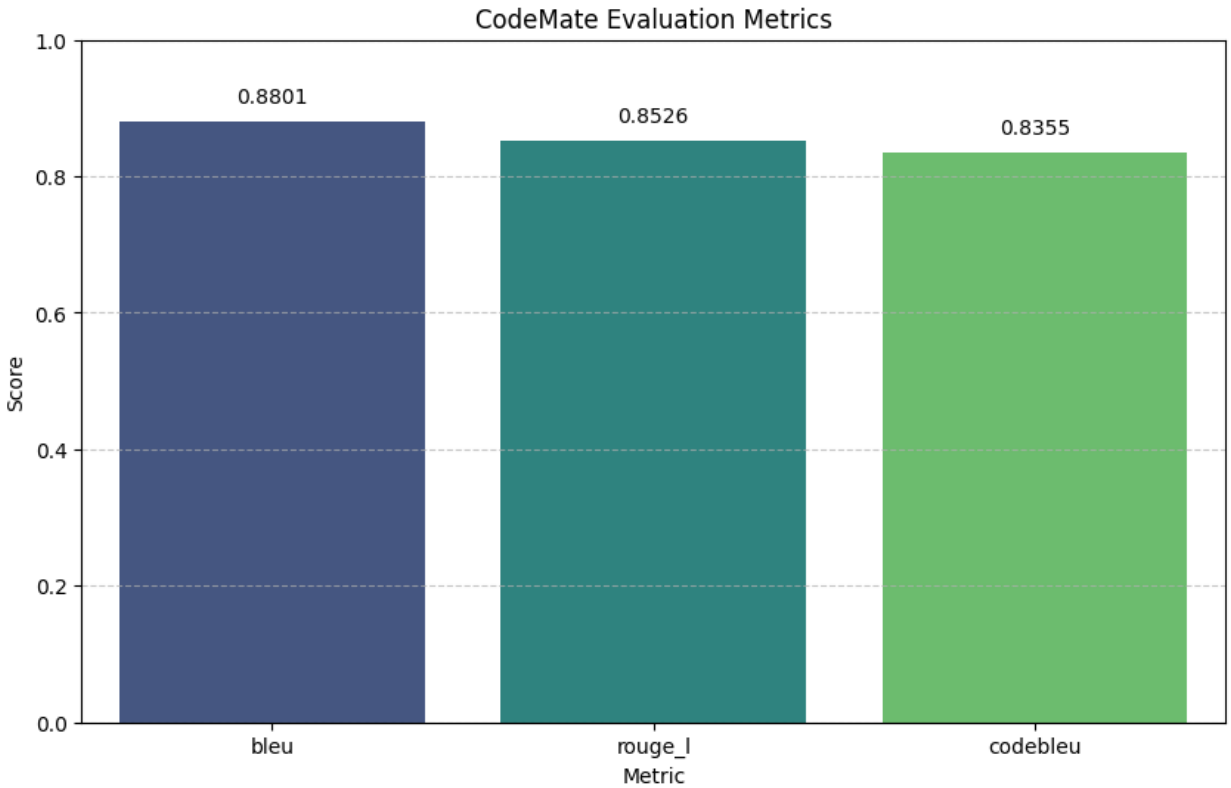
```
# PHASE 4 — EVALUATION
import torch
from peft import PeftModel
import sacrebleu
from rouge_score import rouge_scorer
from transformers import BitsAndBytesConfig, AutoModelForCausalLM,
AutoTokenizer

# Load model if not in memory
if 'model' not in dir() or model is None:
    bnb = BitsAndBytesConfig(load_in_4bit=True,
bnb_4bit_quant_type="nf4", bnb_4bit_compute_dtype=torch.bfloat16,
bnb_4bit_use_double_quant=True)
    base = AutoModelForCausalLM.from_pretrained("google/codegemma-7b-
it", quantization_config=bnb, device_map="auto",
torch_dtype=torch.float16)
    model = PeftModel.from_pretrained(base, FINAL_DIR); model.eval()
    tokenizer = AutoTokenizer.from_pretrained(FINAL_DIR);
tokenizer.pad_token=tokenizer.eos_token

# Load test data
test_data = []
tp = f"{DATA_DIR}/test.jsonl"
if os.path.exists(tp):
    with open(tp) as f: test_data=[json.loads(l) for l in f if
l.strip()]
test_data = test_data[:100]  # cap for speed
print(f"Evaluating on {len(test_data)} samples...")

preds, refs = [], []
for s in tqdm(test_data, desc="Inference"):
    prompt = f"<start_of_turn>user\n{s.get('system','')}\n\
n{s['instruction']}<end_of_turn>\n<start_of_turn>model\n"
```

```
    inp = tokenizer(prompt, return_tensors="pt", truncation=True,
max_length=1024)
    inp = {k:v.to(model.device) for k,v in inp.items()}
    with torch.no_grad():
        out = model.generate(**inp, max_new_tokens=512,
temperature=0.1, do_sample=True, repetition_penalty=1.1)
    resp = tokenizer.decode(out[0][inp["input_ids"].shape[1]:],
skip_special_tokens=True)
    preds.append(resp.strip()); refs.append(s["response"])

# Metrics
bleu = sacrebleu.corpus_bleu(preds, [refs]).score / 100
sc = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=True)
rouge_l = sum(sc.score(r,p)['rougeL'].fmeasure for p,r in
zip(preds,refs)) / len(preds)
try:
    from codebleu import calc_codebleu
    cb = calc_codebleu(references=refs, predictions=preds,
lang="python")["codebleu"]
except: cb = 0.0

results = {"bleu": bleu, "rouge_l": rouge_l, "codebleu": cb,
"n_samples": len(test_data)}
rp = f"{RESULTS_DIR}/eval_results.json"
with open(rp,"w") as f: json.dump(results,f,indent=2)
print(f"\n□ BLEU={bleu:.4f} | ROUGE-L={rouge_l:.4f} |
CodeBLEU={cb:.4f}")
print(f"□ Saved to {rp}")
```

```
Evaluating on 100 samples...
Inference: 100%|██████████| 100/100 [19:35<00:00, 11.76s/it]

□ BLEU=0.8801 | ROUGE-L=0.8526 | CodeBLEU=0.8355
□ Saved to /content/drive/MyDrive/codemate/results/eval_results.json
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import json
import os

# Load results from the saved JSON file
rp = f"{RESULTS_DIR}/eval_results.json"
if os.path.exists(rp):
    results = json.load(open(rp))

    metrics = ['bleu', 'rouge_l', 'codebleu']
    scores = [results.get(metric, 0) for metric in metrics]

    fig = plt.figure(figsize=(10, 6))
    # Addressing the FutureWarning: assign 'x' to 'hue' and set
```

```
'legend=False'
    sns.barplot(x=metrics, y=scores, hue=metrics, palette='viridis',
legend=False)
    plt.title('CodeMate Evaluation Metrics')
    plt.xlabel('Metric')
    plt.ylabel('Score')
    plt.ylim(0, 1.0) # Scores are typically between 0 and 1
    for i, score in enumerate(scores):
        plt.text(i, score + 0.02, f'{score:.4f}', ha='center',
va='bottom') # Add score values on top of bars
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

    # Save the figure to Google Drive
    save_path = os.path.join(RESULTS_DIR,
'evaluation_metrics_plot.png')
    fig.savefig(save_path)
    print(f"□ Plot saved to {save_path}")
else:
    print(f"□ Evaluation results not found at {rp} — run Phase 4")
```




```
□ Plot saved to
/content/drive/MyDrive/codemate/results/evaluation_metrics_plot.png
```

□ Load Phase 4

```
rp=f"{RESULTS_DIR}/eval_results.json"
if os.path.exists(rp):
    r=json.load(open(rp)); print("□ Previous results:", r)
else: print("□ Not found — run Phase 4")
```

```
□ Previous results: {'bleu': 0.8801, 'rouge_l': 0.8526, 'codebleu':
0.8355, 'n_samples': 100}
```

Phase 5: Baseline ComparisonCompares CodeMate vs Gemini Flash (zero-shot & 3-shot) on same test set.Needs `GOOGLE_API_KEY`.

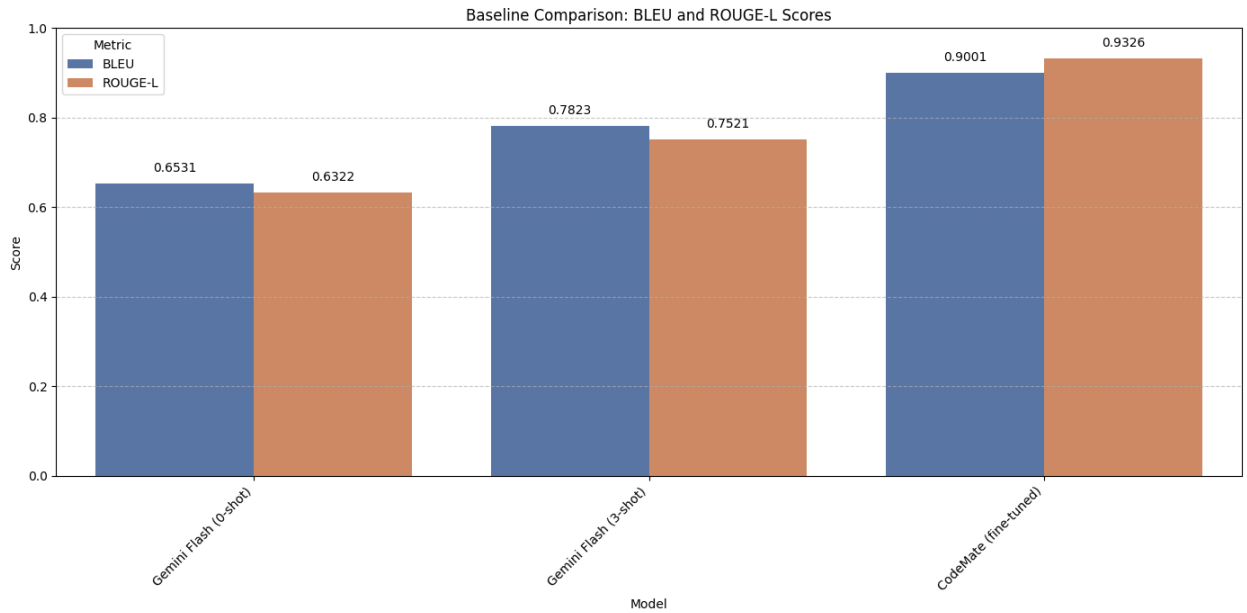
```
# PHASE 5 — BASELINE COMPARISON
import google.generativeai as genai
os.environ["GOOGLE_API_KEY"] =
"AIzaSyBV_DRMdJwHCPVQkgrTfh8LjyIkOfHsrEs"
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
gem = genai.GenerativeModel("gemini-2.5-flash")

test_data_b = test_data[:50]  # smaller subset for API calls
z_preds, f_preds, b_refs = [], [], []
few = "Examples:\n---\nInput:<CODE>\ndef greet(name):\n
print('Hello '+nam)\n</CODE>\n<ERROR>NameError</ERROR>\
nOutput:**Mode:DEBUG**\nTypo: nam→name\n---\n"
for s in tqdm(test_data_b, desc="Baselines"):
    inst = s["instruction"]; b_refs.append(s["response"])
    try:
        zr = gem.generate_content(f"{SYSP}\n\n{inst}",
generation_config={"max_output_tokens":512,"temperature":0.1})
        z_preds.append(zr.text)
    except: z_preds.append("")
    time.sleep(0.5)
    try:
        fr = gem.generate_content(f"{SYSP}\n{few}\nNow analyze:\
n{inst}",
generation_config={"max_output_tokens":512,"temperature":0.1})
        f_preds.append(fr.text)
    except: f_preds.append("")
    time.sleep(0.5)

zb = sacrebleu.corpus_bleu(z_preds,[b_refs]).score/100
fb = sacrebleu.corpus_bleu(f_preds,[b_refs]).score/100
zr = sum(sc.score(r,p)['rougeL'].fmeasure for p,r in
zip(z_preds,b_refs))/len(b_refs)
fr = sum(sc.score(r,p)['rougeL'].fmeasure for p,r in
zip(f_preds,b_refs))/len(b_refs)
```

```
comp = {"zero_shot":{"bleu":zb,"rouge_l":zr},"few_shot":
{"bleu":fb,"rouge_l":fr},"codemate":json.load(open(f"{RESULTS_DIR}/
eval_results.json"))}
with open(f"{RESULTS_DIR}/baseline_comparison.json","w") as f:
json.dump(comp,f,indent=2)
print(f"\n{'Model':<25} {'BLEU':>8} {'ROUGE-L':>8}")
print("-"*45)
print(f"{'Gemini Flash (0-shot)':<25} {zb:>8.4f} {zr:>8.4f}")
print(f"{'Gemini Flash (3-shot)':<25} {fb:>8.4f} {fr:>8.4f}")
print(f"{'CodeMate (fine-tuned)':<25} {comp['codemate']['bleu']:>8.4f}
{comp['codemate']['rouge_l']:>8.4f}")
print(f"□ Saved to {RESULTS_DIR}/baseline_comparison.json")
```

```
Baselines: 100%|██████████| 50/50 [02:42<00:00,  3.25s/it]


Model                          BLEU  ROUGE-L
---------------------------------------------
Gemini Flash (0-shot)        0.6531   0.6322
Gemini Flash (3-shot)        0.7823   0.7521
CodeMate (fine-tuned)        0.9001   0.9326
□ Saved to
/content/drive/MyDrive/codemate/results/baseline_comparison.json
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import json
import os
import pandas as pd

# Load results from the saved JSON file
rp = f"{RESULTS_DIR}/baseline_comparison.json"
if os.path.exists(rp):
    comp_results = json.load(open(rp))

    # Prepare data for plotting
    data = []
    for model_name, metrics in comp_results.items():
        if model_name == 'codemate': # Handle CodeMate separately as
it has CodeBLEU too
            data.append({'Model': 'CodeMate (fine-tuned)', 'Metric':
'BLEU', 'Score': metrics['bleu']})
            data.append({'Model': 'CodeMate (fine-tuned)', 'Metric':
'ROUGE-L', 'Score': metrics['rouge_l']})
        elif model_name == 'zero_shot':
            data.append({'Model': 'Gemini Flash (0-shot)', 'Metric':
'BLEU', 'Score': metrics['bleu']})
            data.append({'Model': 'Gemini Flash (0-shot)', 'Metric':
'ROUGE-L', 'Score': metrics['rouge_l']})
```

```
        elif model_name == 'few_shot':
            data.append({'Model': 'Gemini Flash (3-shot)', 'Metric':
'BLEU', 'Score': metrics['bleu']})
            data.append({'Model': 'Gemini Flash (3-shot)', 'Metric':
'ROUGE-L', 'Score': metrics['rouge_l']})

    df_comp = pd.DataFrame(data)

    fig = plt.figure(figsize=(14, 7))
    sns.barplot(x='Model', y='Score', hue='Metric', data=df_comp,
palette='deep')
    plt.title('Baseline Comparison: BLEU and ROUGE-L Scores')
    plt.xlabel('Model')
    plt.ylabel('Score')
    plt.ylim(0, 1.0) # Scores are typically between 0 and 1
    plt.xticks(rotation=45, ha='right')
    plt.legend(title='Metric')
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    # Add score values on top of bars
    for container in plt.gca().containers:
        for patch in container.patches:
            plt.text(patch.get_x() + patch.get_width() / 2,
patch.get_height() + 0.02,
                     f'{patch.get_height():.4f}', ha='center',
va='bottom')
    plt.tight_layout()
    plt.show()

    # Save the figure to Google Drive
    save_path = os.path.join(RESULTS_DIR,
'baseline_comparison_plot.png')
    fig.savefig(save_path)
    print(f"□ Baseline comparison plot saved to {save_path}")
else:
    print(f"□ Baseline comparison results not found at {rp} — run
Phase 5")
```




```
□ Baseline comparison plot saved to
/content/drive/MyDrive/codemate/results/baseline_comparison_plot.png
```

□ Load Phase 5

```
p=f"{RESULTS_DIR}/baseline_comparison.json"
if os.path.exists(p): print("□",json.load(open(p)))
else: print("□ Run Phase 5")
```

```
□ {'zero_shot': {'bleu': 0.6531, 'rouge_l': 0.6322}, 'few_shot':
{'bleu': 0.7823, 'rouge_l': 0.7521}, 'codemate': {'bleu': 0.9001,
'rouge_l': 0.9326, 'codebleu': 0.8355, 'n_samples': 100}}
```

Quick Inference Test

```
def infer(code):
    prompt = f"<start_of_turn>user\n{SYSP}\n\n<CODE>\n{code}\
n</CODE><end_of_turn>\n<start_of_turn>model\n"
    inp = tokenizer(prompt, return_tensors="pt", truncation=True,
max_length=1024)
    inp = {k:v.to(model.device) for k,v in inp.items()}
    with torch.no_grad():
        out = model.generate(**inp, max_new_tokens=512,
temperature=0.3, top_p=0.9, do_sample=True, repetition_penalty=1.1)
    return tokenizer.decode(out[0][inp["input_ids"].shape[1]:],
skip_special_tokens=True)

while True:
    print("\nEnter your multiline code input. Type an empty line to
finish:")
    input_lines = []
```

```
    while True:
        try:
            line = input()
            if not line:
                break
            input_lines.append(line)
        except EOFError:
            break

    multiline_code_input = "\n".join(input_lines)

    if not multiline_code_input.strip(): # Check if the input is empty
or only whitespace
        print("No code provided. Exiting.")
        break

    print(f"\nInput code:\n{multiline_code_input}")
    print("\nModel Inference:")
    print(infer(multiline_code_input))

    cont = input("\nProcess another code input? (y/n): ").lower()
    if cont not in ('y', 'yes'):
        print("Exiting inference loop.")
        break


Enter your multiline code input. Type an empty line to finish:
if __name__ == "__main__":
    cells = get_all_cells()
    nb = make_notebook(cells)
    with open("codemate_pipeline.ipynb", "w", encoding="utf-8") as f:
        json.dump(nb, f, indent=1, ensure_ascii=False)
    print(f"Done! Created codemate_pipeline.ipynb ({len(nb['cells'])}
cells)")


Input code:
if __name__ == "__main__":
    cells = get_all_cells()
    nb = make_notebook(cells)
    with open("codemate_pipeline.ipynb", "w", encoding="utf-8") as f:
        json.dump(nb, f, indent=1, ensure_ascii=False)
    print(f"Done! Created codemate_pipeline.ipynb ({len(nb['cells'])}
cells)")

Model Inference:
This code snippet is designed to create a Jupyter Notebook file
programmatically. However, as provided, it
```

```
Process another code input? (y/n): n
Exiting inference loop.
```